

Author: Andrea Mussari

1)

The MAX-CUT problem seeks to partition the vertices V of a graph $G = (V, E)$ into two sets, say S and $V \setminus S$, such that the number of edges connecting a vertex in S to a vertex in $V \setminus S$ (the cutset $E(S, V \setminus S)$) is maximized. Conceptually, this can be seen as finding a 2-coloring of the graph's vertices that maximizes the number of edges with endpoints of different colors. MAX-CUT is known to be an NP-hard problem.

Randomization is a powerful tool in algorithm design, and a simple approach for MAX-CUT is to assign each vertex to one of the two sets randomly. This is akin to a random 2-coloring.

The algorithm is as follows:

- Given an unweighted, undirected graph $G = (V, E)$.
- Initialize two empty sets, S_1 and S_2 .
- For each vertex $v \in V$:
 - Assign v to set S_1 with probability $1/2$.
 - Assign v to set S_2 with probability $1/2$.
 - These assignments are made **independently** for each vertex.
- After all vertices are assigned, the cut is defined by the partition (S_1, S_2) .
- Calculate the size of the cut, which is the number of edges $u, v \in E$ where $u \in S_1$ and $v \in S_2$, or $u \in S_2$ and $v \in S_1$.
- Return the size of this cut.

This algorithm involves iterating through all vertices and making a random choice for each. Then, counting the edges in the cut requires iterating through the edges. This procedure is clearly **polynomial time**, running in $O(|V| + |E|)$ time.

2)

To prove that this algorithm provides a 2-approximation *in expectation*, we analyze the expected size of the cut found. We use the technique of **random indicator variables** and the property of **linearity of expectation**.

Let OPT be the size of the maximum cut in graph G . Our goal is to show that the expected size of the cut produced by the randomized algorithm, $E[|E(S_1, S_2)|]$, is at least $\frac{1}{2}OPT$.

1. **Define Indicator Variables:** For each edge $e = u, v \in E$, define a random indicator variable X_e : $X_e = 1$ if the edge e is in the cut (i.e., its endpoints u and v are in different sets, S_1 and S_2). $X_e = 0$ otherwise (if u and v are in the same set).
2. **Express Cut Size:** The total number of edges in the cut $E(S_1, S_2)$ is the sum of the indicator variables for all edges in the graph: $|E(S_1, S_2)| =$

$$\sum_{e \in E} X_e.$$

3. **Calculate Expected Cut Size:** By the linearity of expectation, the expectation of the sum is the sum of the expectations: $E[|E(S_1, S_2)|] = E[\sum_{e \in E} X_e] = \sum_{e \in E} E[X_e]$.
4. **Calculate Expected Value of Indicator Variable:** For an indicator variable X_e , its expected value $E[X_e]$ is equal to the probability that the event $X_e = 1$ occurs. The event $X_e = 1$ happens if the endpoints u and v of edge $e = u, v$ are assigned to different sets. Since each vertex is assigned to S_1 or S_2 independently with probability $1/2$, there are four possible outcomes for the assignment of u, v , each with probability $(1/2) \times (1/2) = 1/4$:
 - $u \in S_1, v \in S_1$: probability $1/4$ (edge not in cut).
 - $u \in S_1, v \in S_2$: probability $1/4$ (edge in cut).
 - $u \in S_2, v \in S_1$: probability $1/4$ (edge in cut).
 - $u \in S_2, v \in S_2$: probability $1/4$ (edge not in cut).

The event that u and v are in different sets occurs with probability $1/4 + 1/4 = 1/2$. Thus, for any edge $e \in E$, $Pr[X_e = 1] = 1/2$. The expected value of X_e is $E[X_e] = Pr[X_e = 1] = 1/2$.

5. **Complete Expectation Calculation:** Substituting $E[X_e] = 1/2$ into the sum: $E[|E(S_1, S_2)|] = \sum_{e \in E} (1/2) = \frac{1}{2}|E|$.

The expected size of the cut found by this randomized algorithm is half the total number of edges in the graph.

6. **Relate to Optimal:** The size of the maximum cut OPT cannot exceed the total number of edges $|E|$ ($OPT \leq |E|$). Therefore, the expected cut size satisfies: $E[|E(S_1, S_2)|] = \frac{1}{2}|E| \geq \frac{1}{2}OPT$.

This shows that, **in expectation**, the simple randomized algorithm finds a cut whose size is at least half the size of the maximum cut, making it a 2-approximation in expectation for the MAX-CUT problem.

3)

While the simple randomized algorithm provides a good guarantee on the expected cut size, a single execution might yield a cut much smaller than the expectation. For algorithms that have a probability of producing a “bad” result (often called Monte Carlo algorithms), a standard technique to reduce the probability of failure is to **repeat the algorithm multiple times** and combine the results.

For a maximization problem like MAX-CUT, if we repeat the simple randomized algorithm N times independently, we can then **output the maximum cut size found across these N executions**. If a single run fails to produce a cut of a certain quality (e.g., at least a fraction c of the optimum) with probability

p , then repeating it N times independently means the probability that *all* N runs fail is p^N . This probability decreases **exponentially** with N , allowing us to achieve an arbitrarily low error probability δ by choosing N sufficiently large.

Chernoff bounds (CBs) are a key tool in the analysis of randomized algorithms, particularly for bounding the probability that a sum of independent random variables deviates significantly from its expected value. While the simple MAX-CUT algorithm result we proved relates to the expectation of a *single* run's outcome (which is a sum of indicator variables), CBs are typically used when we have a sum or average of *multiple independent trials* of the random experiment.

In the context of repeating the MAX-CUT algorithm:

- Each of the N runs produces a cut size, say $C_1, C_2, \dots, C_N$. These are independent random variables with the same expectation $E[C_i] = \frac{1}{2}|E|$.
- We are interested in the maximum of these values: $C_{\max} = \max(C_1, \dots, C_N)$.
- Although directly applying standard CBs to bound the deviation of the *maximum* is not as straightforward as bounding a sum or average, CBs (or related concentration bounds like Azuma-Hoeffding bound, which is mentioned in the sources and similar to CBs) are used in algorithm analysis to quantify how quickly the sum or average of independent random variables concentrates around the expectation. This concentration property is what allows repeating experiments to be effective.
- Specifically, if we can bound the probability that a single run's outcome C_i is significantly *below* its expectation or below some desirable threshold (e.g., $\Pr[C_i < \alpha \cdot OPT]$ for some $\alpha > 1/2$) by some probability $p < 1/2$, then the probability that *all* N runs produce a result below this threshold is p^N . To make this failure probability p^N smaller than a desired δ , we need $N \log p < \log \delta$, or $N > \frac{\log(1/\delta)}{\log(1/p)}$. Since p is bounded, N can be chosen as $O(\log(1/\delta))$ (possibly with polynomial factors in problem size depending on how p is bounded). CBs would be used to provide the upper bound p on the probability of a single run being “bad”.

By repeating the simple randomized MAX-CUT algorithm $N = O(\log(1/\delta))$ times and taking the maximum cut found, we can obtain a cut whose size is, with high probability (e.g., $1 - \delta$), guaranteed to be closer to the expected value (and thus to half the optimum) than a single run might provide. This allows boosting the probability of getting a result close to the expected 2-approximation guarantee. This approach of repetition to reduce error probability is demonstrated for other randomized algorithms in the sources, such as the MIN-CUT algorithm.